

AI-Powered Snapshot Lifecycle Management and Storage Optimization System

Ms Ramala T

Assistant Professor

Computer Science and Engineering,
Rajalakshmi Engineering College,
Chennai, India

ramala.t@rajalakshmi.edu.in

Rukesh Kumaran M

Student

Computer Science and Engineering,
Rajalakshmi Engineering College,
Chennai, India

230701271@rajalakshmi.edu.in

Sai Aravind S

Student

Computer Science and Engineering,
Rajalakshmi Engineering College,
Chennai, India

230701275@rajalakshmi.edu.in

Sharukesh S

Student

Computer Science and Engineering,
Rajalakshmi Engineering College,
Chennai, India

230701308@rajalakshmi.edu.in

Siddarth Sakthi M

Student

Computer Science and Engineering,
Rajalakshmi Engineering College,
Chennai, India

230701314@rajalakshmi.edu.in

Abstract - Most cloud disaster-recovery configurations currently employ snapshots with static intervals and retention durations. A common default is 1/hour snapshots retained for a month. These practices are often ill-suited to the dynamic behavior of the workloads they are supposed to protect, and provide no guidance for fulfilling specific requirements during a restore. In this paper, we present a snapshot lifecycle control layer, implemented on top of the Azure snapshot and backup services, that learns when to trigger new snapshots, and when a given snapshot can be deleted, so that the cost-benefit tradeoff during a disaster recovery is optimized per workload. Our approach is inspired by a risk-adaptive interval method previously introduced for ML training checkpoints on spot/preemptible VMs. We adapt the method to capture snapshot timing decisions while taking into account forecasted risk, and enrich it with a policy-enforced cost model to derive a closed-form expression for the optimal interval between consecutive snapshots. While the original method only concerned itself with the timing of snapshot creations, we additionally derive a closed-form expression for a retention policy maximizing the information retained while minimizing storage costs. To learn the retention policy, we perform a weighted analysis of the storage cost vs. the retention value of a given snapshot, and derive the optimal storage duration using the Lambert W function. For comparison purposes, we additionally train a supervised XGBoost classifier on the same telemetry trace, to recover the cutoff decision boundaries learned by our analytical method. The evaluation is performed on real production telemetry from the GWA-T-12 Bitbrains trace, augmented with synthetic recovery tiers targeting a range of desired RPO, RTO and criticality values, followed by a deterministic safety pol-

icy engine (to prevent accidental deletions based on any model's recommendation), and dry-run execution by default. The comparison methodology is also evaluated on snapshot cost, count of redundant/unnecessary snapshots, RPO violation rate, recovery coverage and false deletion rate.

Keywords: Cloud Snapshot Lifecycle Management, Disaster Recovery, Azure VM, Risk-Adaptive Snapshot Scheduling, Cost Optimization, Snapshot Retention, XGBoost, RPO, RTO, Lambert W, Safety Policy Engine

I. INTRODUCTION

Cloud infrastructure has become the backbone of modern business. The responsibility of ensuring that virtual machines are recoverable in the event of a failure falls to the infrastructure team, and snapshots are the most common method employed for this; providers such as Azure have already established a native snapshot and Backup service. The issue with the latter is how it is applied: most often, a fixed schedule and retention policy is applied, and then left to wither. A dev server is snapshotted as frequently as a busy production database, incurring unnecessary costs, and a sudden burst of writes is too late in between two scheduled captures to be safely recovered from. Old snapshots accumulate, months after they are useful. Progress in time series forecasting, cost-based optimisation and machine learning presents new opportunities to tackle these decisions in a more adaptive manner. With work on ML training checkpointing over preemptible cloud VMs, we find that a risk forecast plus a closed-form cost model provides an optimal interval for protection actions, but this has remained con-

fined to this domain, and only answers the question of when to create a snapshot, not when one can be deleted.

The proposed system addresses the latter question, providing an orchestration layer one level above Azure’s native snapshot mechanisms, making two decisions: whether to create a snapshot now or leave things be, and whether to retain or delete an existing one. For the former decision, a lightweight exponential-smoothing forecaster derives near-term disk change volatility, and a closed-form square-root interval is clamped to RPO limits. For the latter decision, a matching closed-form model weighs linear storage cost against exponentially-decaying value of an ageing snapshot, solving for crossover age with the Lambert W function while respecting a mandatory policy floor. A supervised XGBoost classifier, trained on labels from the closed-form threshold, is compared against the analytically-derived optimum. Both are evaluated on real production telemetry from the Bitbrains trace, and simulated recovery tiers representing varying RPO, RTO and criticality levels. Since a wrongly deleted snapshot cannot be recovered, a deterministic Safety Policy Engine validates all recommendations prior to their being passed to the cloud—dry-run is the default mode, and any uncertainty or API error results in no deletion. The intention of this work is to move from static, human-configured backup rules to a workload aware, cost-efficient and safe approach that decides when to protect data, when to delete snapshots, and how to continue to improve over time.

II. LITERATURE REVIEW

The choice of a risk-adaptive interval for a protective action has a long prehistory in reliability engineering. Young [1] showed that the first-order approximation of the optimum checkpoint interval was achieved by equalizing the cost of writing a checkpoint with the work performed since the last one. This led Daly [2] to propose a higher-order approximation, which has since become a textbook formula. Benoit et al. [3] surveyed the accuracy of the Young/Daly-style cost function in various application domains. All three works demonstrate that the optimal interval is computable: in fact, they suggest the square-root function which underlies the snapshot creation model used in this project. However, these results were obtained for checkpointing in high-performance computing applications. For cloud VM snapshots, the question of retention intervals has to be addressed separately.

Recent work by Desai et al. [4] extends the application of risk-adaptive checkpointing to large-model training on preemptible cloud VM instances. Their main contribution is the forecast of preempted training jobs based on historical data, which allows for tuning the checkpointing interval

according to a mathematical model of overhead vs recovery cost. This justifies the adaptive approach, but their analysis begins and ends with the creation of backups. Neither storage costs nor the diminishing value of an older recovery point are taken into account, and no retention decisions are made. This project adopts the forecast + closed-form risk model of Desai et al. for snapshot creation but adds a separate component for retention.

The cloud platforms have first-class tools for managing backup snapshots already today. Azure Backup allows the administrator to define a policy with a schedule and retention range, for example, hourly checkpoints within the Enhanced policy [5]. The same documentation recommends adjusting schedule and retention based on workload type and criticality, but the tuning is manual. Another native solution is Azure Blob lifecycle management, which can delete snapshots in accordance with predefined rules, for instance, at the end of their retention period [6]. These tools are reliable and form the competition landscape, but their rules are static and cannot adapt to workload changes. The present project works one level higher: it defines the conditions under which the platform’s automation should be triggered.

Some researchers have explored the option of machine learning (ML) for snapshot scheduling. Khan et al. [7] describe the Adaptive Predictive Snapshot Scheduler for storage area networks. Their solution uses Random Forest classification and Gradient Boosting regression to predict the optimal time for a snapshot. In their experiments, it has shown lower storage overhead and better adherence to recovery point objectives (RPO) than traditional periodic and threshold-based approaches. It is similar to the current project in that it tries to learn the best moment for a snapshot. However, Khan et al. used standard ML tools, while this project learns a closed-form analytical expression for the retention crossover point. More importantly, they trained their models on synthetic data and did not address the question of cost-optimality, while the opposite is true for this project.

Any ML component requires a dataset for training and evaluation. In particular, snapshot scheduling research often uses trace-driven simulations. However, most publications only use workloads simulated specifically for the task. The GWA-T-12 Bitbrains traces [12] originate from the production datacenter of a managed-hosting company and represent a class of business-critical workloads. They lack direct information about recovery objectives or criticality, which is typical of real-world traces. This project uses the trace set as a starting point for synthetic recovery requirement generation. It also implements a deterministic Safety Policy Engine to evaluate deletion recommendations for each snapshot.

To summarize, each component of the current project has predecessors in the literature. Classical interval formulas, risk-adaptive checkpointing, native cloud tools for snapshots, ML-based scheduling and trace-driven evaluation have all been explored. However, no previous work has combined these areas. In particular, none of the papers mentioned above have addressed the question of retention in terms of cost and risk tradeoffs. This project fills this gap by proposing a snapshot lifecycle management framework, which operates on real cloud data and adheres to safety-critical constraints.

III. METHODOLOGY

The system is structured in three functional layers and consists of a set of tightly coupled modules that form an end-to-end snapshot lifecycle pipeline, with data collection segregated from risk forecasting, forecasting from cost-based decision making, and every decision passing through a deterministic safety layer before reaching the cloud. The Data Collection and Dataset modules collate infrastructure metadata, live telemetry and real workload traces. The Feature Engineering and Risk Forecasting module transforms this raw data into risk signals. The Cost Optimizer, the XGBoost model and the Decision Engines are responsible for the two core decisions, while the Safety Policy Engine, Azure Automation, the Dashboard and the Evaluation module control execution and evaluate the results. The architecture has three primary layers; 1. Data and Telemetry, 2. Decision Intelligence, and 3. Safety and Execution. The Data and Telemetry layer collates snapshot metadata, Azure Monitor metrics and the historical Bitbrains dataset, and prepares features from them. The Decision Intelligence layer forecasts risk, derives the optimal snapshot interval and retention age in closed form, and compares these with a learned classifier. The Safety and Execution layer evaluates each recommendation against RPO, RTO and retention rules and executes approved actions in dry-run or live mode via the Azure API. Every decision is logged, enabling audit and comparison with the fixed-schedule baseline.

DATA COLLECTION AND DATASET PACKAGE

The Data Collection and Dataset Package integrates M3 Snapshot Metadata Collector, M4 Azure Monitor Metrics Collector and M5 Dataset Generation. M3 leverages the Azure compute SDK to read snapshot age, disk size, incremental status, source disk and tags from the live environment. M4 utilizes the Azure monitor SDK to collate CPU utilization, disk read and write bytes, disk IOPS and network activity from a real Azure VM running a generated test workload. M5 builds the historical training dataset

from a subset of 50 to 100 VMs in the GWA-T-12 Bitbrains fastStorage trace, which captures real production workload behaviour. As recovery requirements are business decisions and not measurable signals, each selected VM is assigned once to one of three simulated policy tiers, high, medium or low criticality, with corresponding RPO and RTO values. Together, these modules provide the system with both live data for validation and realistic data for training and evaluation.

RISK FORECASTING PACKAGE

The Risk Forecasting Package contains M6 Feature Engineering, tasked with transforming raw telemetry into inputs for use by the decision models. With the disk activity history, a lightweight exponential-smoothing forecaster estimates near-term disk change-rate volatility and produces a predicted change rate and a volatility score. This forecaster deliberately avoids the heavy dependencies of the SARIMA model used in the base paper, enabling speed and isolation. These risk values, along with the tier's RPO, RTO and criticality, form the common feature set used by both the closed-form model and the learned model. This package enables the system to transition from fixed schedules to decisions that respond to how the workload is behaving.

COST-OPTIMAL DECISION PACKAGE

The Cost-Optimal Decision Package is centered on M6.5 Cost Optimizer, which contains the two main contributions of this work. For snapshot creation, the optimizer balances snapshot overhead against the expected loss from unprotected changes and derives a closed-form square-root interval. This interval is then clamped between the minimum and maximum allowed values and can never exceed the RPO of the workload tier. For retention, it compares a storage cost that increases linearly with snapshot age against a recovery value that decays exponentially, and solves for the crossover age using the Lambert W function. The mandatory policy floor is then applied, ensuring a snapshot is never marked for deletion earlier than the policy allows. The creation side is adapted from the checkpoint-interval method of the base paper, while the retention side is an extension that the source method does not possess.

The snapshot interval balances snapshot overhead against expected loss from unprotected changes. The project adopts the following cost model:

$$C(T) = \frac{C_s}{T} + \frac{\lambda LT}{2} \quad (1)$$

where T is the snapshot interval, C_s is snapshot overhead, λ is the predicted disaster/risk rate, and L is the loss cost per minute of unprotected change. Differentiating with respect

to T gives the closed-form optimum

$$T^* = \sqrt{\frac{2C_s}{\lambda L}}. \quad (2)$$

The resulting interval is projected onto the feasible interval $[T_{\min}, \min(T_{\max}, \text{RPO})]$, ensuring that the recommendation cannot exceed the workload's hard RPO limit.

For retention, storage cost increases approximately linearly with snapshot age while recovery value decays exponentially. The model is defined as

$$S(a) = sa, \quad R(a) = wL_d e^{-\gamma a}. \quad (3)$$

The crossover age satisfies

$$sa = wL_d e^{-\gamma a}, \quad (4)$$

and is solved with the Lambert W function as

$$a^* = \frac{1}{\gamma} W\left(\frac{\gamma w L_d}{s}\right). \quad (5)$$

A mandatory policy floor h is then imposed, giving the effective threshold

$$A^* = \max(h, a^*). \quad (6)$$

LEARNED MODEL AND DECISION ENGINE PACKAGE

The Learned Model and Decision Engine Package combines M7 XGBoost Training with M8 and M9, the Decision Engines for snapshot creation and snapshot retention. The XGBoost classifier is trained on the same feature set, with CREATE or WAIT labels generated from the closed-form threshold rather than hand-written rules. Isolation Forest can be optionally added as an anomaly signal for sudden workload changes. For every decision, the engines produce two independent recommendations, one from the cost optimizer and one from XGBoost, enabling comparison between the derived optimum and the learned approximation. The final recommendations are CREATE or WAIT for creation and KEEP or DELETE for retention.

SAFETY, AUTOMATION AND EVALUATION PACKAGE

The Safety, Automation and Evaluation Package joins M10 Safety Policy Engine, M11 Azure Automation and M12 Dashboard with M14 Evaluation, to control execution and evaluate the results. The Safety Policy Engine is a deterministic gate that verifies RPO and RTO compliance, minimum recovery-point coverage and retention floors, ensuring no action can be directed to Azure by AI or optimizer. Dry-run is the default mode, live execution requires explicit configuration and confirmation, and any Azure API error or

uncertainty results in no deletion. M11 executes approved actions through DefaultAzureCredential, with no hardcoded secrets, and M12 presents decisions and metrics in a Streamlit dashboard. M14 compares the fixed-schedule baseline, the cost-optimal model and the XGBoost model on identical data, using storage cost, snapshot counts, unnecessary snapshots, RPO violations, recovery coverage and false deletions or creations. This package provides the system with a loop where workload conditions are observed, recommendations made, checked for safety, the approved steps executed, and the result measured.

SYSTEM ARCHITECTURE

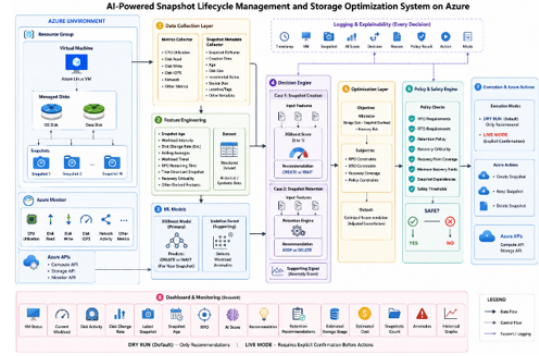


Figure 3.1. Architecture Diagram of the System

IV. RESULT AND ANALYSIS

The first analysis explores the Bitbrains VMs' workload data that underlies all further comparisons. In particular, disk change-rate trends are evaluated in terms of burstiness, volatility, and the distribution across the three recovery tiers. This serves to characterize the input data in detail, establish whether the workloads are sufficiently diverse to allow isolation of an adaptive approach, and determine whether exponential-smoothing forecasts capture the relevant change patterns for the risk signals to be informative to downstream processes.

The second analysis focuses on snapshot creation, specifically, the recommendations made by the cost optimizer. First, the closed-form interval is compared to the fixed hourly baseline in terms of the number of snapshots, storage costs, the number of low-value (inefficient) snapshots, and the number of RPO violations per tier. Second, the findings are broken down by recovery tier to determine whether high-criticality workloads indeed involve shorter intervals than the low-criticality ones. This analysis aims to demonstrate that risk-adaptive intervals better protect the data governed by the higher recovery targets while still reducing the resource expenditures and operational complex-

ity caused by frequent snapshots. Finally, the optimizer’s ability to clamp down on the intervals to meet the RPO targets is evaluated.

The third and fourth analyses examine the retention policy from two angles: the closed-form crossover age with the policy floor and the learned model. The former is compared to the fixed thirty-day baseline in terms of the total coverage, the storage allocated to the retained snapshots, and the number of false deletions, i.e., the number of snapshots deleted despite their potential value to the recovery process. Next, the XGBoost classifier’s performance is similarly evaluated using accuracy, precision, recall, F1-score, ROC-AUC, and a confusion matrix that identifies which CREATE and WAIT decisions were classified correctly or incorrectly. These two approaches to retention policy optimization are subsequently compared to the fixed baseline in terms of the same metrics, with the emphasis on the number of unnecessary (low-value) snapshots. The agreement between the closed-form and learned models serves as an additional validation metric, while their performance relative to the fixed baseline indicates whether there is value in pursuing a model-based approach to retention.

The fifth analysis explores the Safety Policy Engine by evaluating the recommendations made by the optimizer and the XGBoost classifier. In particular, the recommendations are compared to the RPO, RTO, coverage, and retention-floor limits to count the number of violations of each across the three recovery tiers. Furthermore, the analysis checks whether the default dry-run mode is indeed disabled and whether any of the Azure API errors or uncertainties prevent the system from deleting the snapshots as recommended. Ultimately, the analysis attempts to demonstrate that the system is safe in its current configuration, i.e., no recommendation, including those violating the safety policies, triggers any action.

The sixth analysis integrates the findings of the previous parts to demonstrate the value of the combined approach to snapshot lifecycle management. In particular, the scenarios involving the high and low volatility of workload change rates are compared across the three recovery tiers in terms of storage costs, RPO violations, coverage, and the number of unnecessary snapshots. All individual decisions are broken down by their input variables, process recommendations, final policy outcome, and execution mode to enable further auditing and analysis. Overall, the six sets of findings are intended to illustrate how the proposed approach to adaptive and automated backup snapshot management can replace the traditional manual and rigid scheduling methods, reducing costs while complying with the most demanding recovery requirements. The quantitative results of the analysis will become available after completing the M14 assessment.

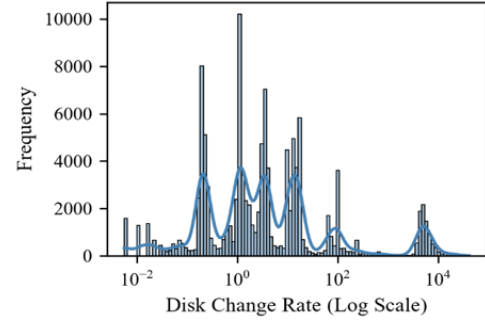


Figure 4.1. Workload Volatility and Snapshot Change-Rate Analysis

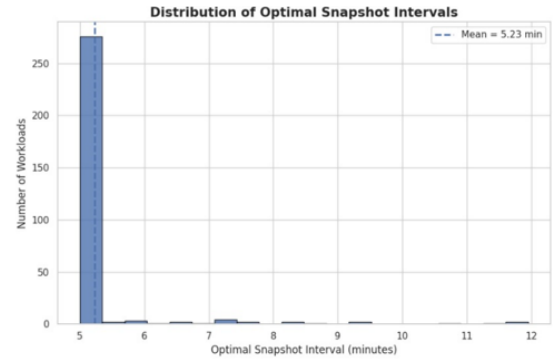


Figure 4.2. Snapshot Creation Optimization - Baseline vs. Cost-Optimal

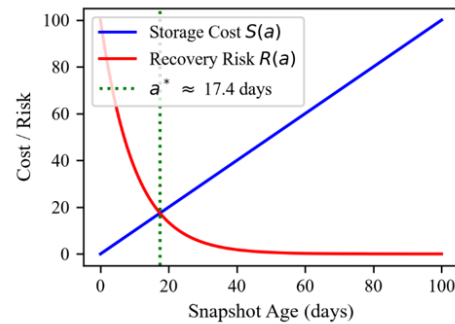


Figure 4.3. Snapshot Retention Optimization and Recovery Coverage

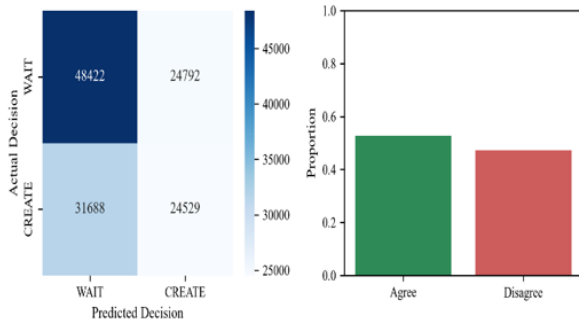


Figure 4.4. XGBoost Decision Comparison and Safety-Gate Analysis

V. CONCLUSION

The proposed solution represents an intelligent orchestration layer for VM snapshot lifecycle management that sits atop Azure’s native technologies, with the ability to leverage risk forecasting, a closed-form snapshot interval, closed-form retention model, learned XGBoost comparison, and deterministic Safety Policy Engine to determine what and when to snapshot, and what and when to delete the old snapshots in accordance with workload behavior and recovery-tier-specific requirements. With the combined insights of production Bitbrains telemetry and policy-controlled safeguards, it becomes possible to reduce storage costs and protect data with confidence. Ultimately, the solution would allow organizations to shift from rigid, policy-driven data backup to dynamic, workload-aware and safe snapshot management.

REFERENCES

- [1] J. W. Young, “A first order approximation to the optimum checkpoint interval,” *Communications of the ACM*, vol. 17, no. 9, pp. 530–531, 1974. doi:10.1145/361147.361115
- [2] J. T. Daly, “A higher order estimate of the optimum checkpoint interval for restart dumps,” *Future Generation Computer Systems*, vol. 22, no. 3, pp. 303–312, 2006. doi:10.1016/j.future.2004.11.016
- [3] A. Benoit et al., “Checkpointing à la Young/Daly: An Overview,” *Int. Conf. on Contemporary Computing (IC3)*, 2022.
- [4] O. Desai, S. Pei, J. Bhimani and B. S. Kim, “Cost-Efficient Training and Checkpointing for Large Models on Pre-emptible Cloud VMs,” *EuroMLSys ’26*, pp. 31–40, 2026. doi:10.1145/3805621.3807617
- [5] Microsoft Learn, “Back up Azure VMs with the En-

hanced policy” and “Guidance and best practices,” Azure Backup documentation. learn.microsoft.com/en-us/azure/backup/

- [6] Microsoft Learn, “Azure Blob Storage lifecycle management overview.” learn.microsoft.com/en-us/azure/storage/blobs/lifecycle-management-overview
- [7] S. Y. Khan, R. S. A. Larik, A. Bachayo, B. Ali and Kanwal, “ML-Based Adaptive Backup Snapshot Scheduling for Storage Area Networks,” *Lahore Garrison Univ. Res. J. Comput. Sci. Inf. Technol.*, vol. 10, no. 1, pp. 36–50, 2026. doi:10.54692/lgurjcsit.v10i1.203
- [8] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., OTexts, 2021. otexts.com/fpp3
- [9] R. M. Corless, G. H. Gonnet, D. E. G. Hare, D. J. Jeffrey and D. E. Knuth, “On the Lambert W function,” *Advances in Computational Mathematics*, vol. 5, pp. 329–359, 1996.
- [10] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” *Proc. 22nd ACM SIGKDD*, pp. 785–794, 2016. doi:10.1145/2939672.2939785
- [11] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation Forest,” *Proc. IEEE ICDM*, pp. 413–422, 2008. doi:10.1109/ICDM.2008.17
- [12] S. Shen, V. van Beek and A. Iosup, “Statistical Characterization of Business-Critical Workloads Hosted in Cloud Datacenters,” *Proc. IEEE/ACM CCGrid*, pp. 465–474, 2015. doi:10.1109/CCGrid.2015.60